

YOLO EN VISIÓN ARTIFICIAL PARA LA INGENIERÍA

YOLO en Ingeniería de Visión Artificial

Entrenamiento y Aplicaciones Avanzadas

Autor: Alejandro Silvestri

1.00

Instalación y Uso de Ultralytics YOLO

- **Biblioteca unificada** y estandarizada para visión artificial.
- **Instalación rápida** y directa: `pip install ultralytics`
- **Soporte dual y flexible**: Interfaz de línea de comandos (CLI) y API nativa de Python.

Terminal CLI

```
# Instalar paquete de forma global
$ pip install ultralytics

# Ejecutar inferencia desde comandos
$ yolo predict model=yolov8n.pt source="img.jpg"
```

Python API

```
from ultralytics import YOLO

# Cargar un modelo preentrenado
model = YOLO("yolov8n.pt")

# Inferencia nativa del framework
results = model("imagen.jpg")
```

Evolución de la Arquitectura YOLO

- **Origen:** YOLOv1 introducido por Joseph Redmon en 2015.
- **Transición técnica:** Migración del framework Darknet (C) hacia PyTorch.
- **Estado actual:** Ecosistema consolidado por Ultralytics con arquitecturas avanzadas (YOLOv8 y YOLO11).



Versiones de YOLO

- **YOLO:** You Only Look Once, de Joseph Redmon en 2015
- **YOLO 9000:** 9000 categorías
- **YOLO 3:** renombraron las anteriores como YOLO 1 y YOLO 2
- **YOLO 4:** de Alexey Bochkovskiy, sucesor técnico en la comunidad de código abierto
- **YOLO 9000:** 9000 categorías
- **YOLO 9000:** 9000 categorías
-

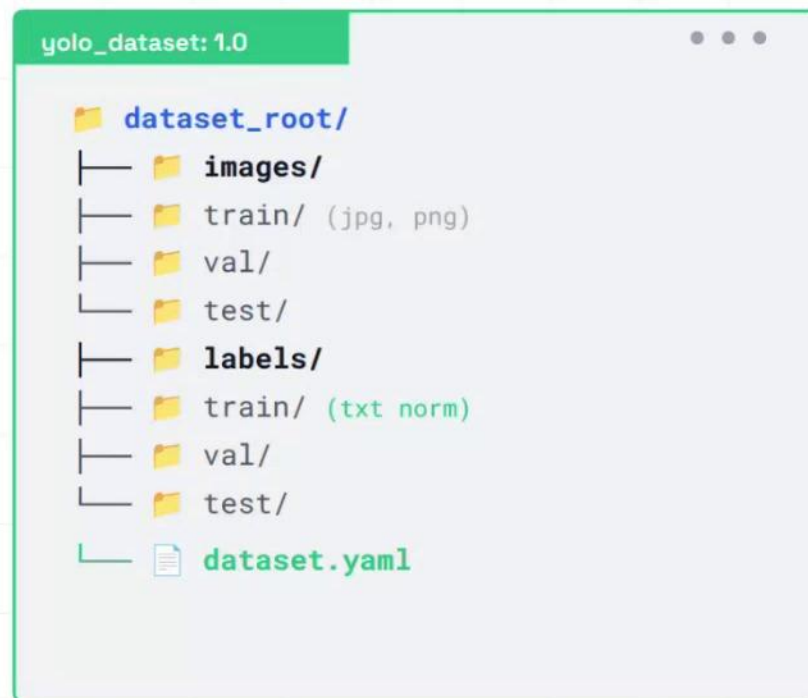
Ultralytics

- **Marca YOLO:** Ultralytics registró la marca YOLO, permite su uso libre en publicaciones académicas; lanzó varias versiones sin paper
- **YOLOv5**
- **YOLOv8**
- **YOLO11**
- **YOLO26**
- **YOLO 27**

Estructuración del Dataset

Requisitos del Dataset

- **Jerarquía estricta:** Carpetas separadas para `images/` y `labels/` (divididas en train, val, test).
- **Formato de etiquetas:** Archivos `.txt` con coordenadas normalizadas para cada objeto detectado.
- **Configuración:** El archivo `dataset.yaml` mapea las rutas y define con exactitud las clases del modelo.



Entrenamiento Acelerado en Google Colab

- **Requisito fundamental:** Entorno de ejecución con aceleración de hardware (GPU).



colab_training_yolo.ipynb

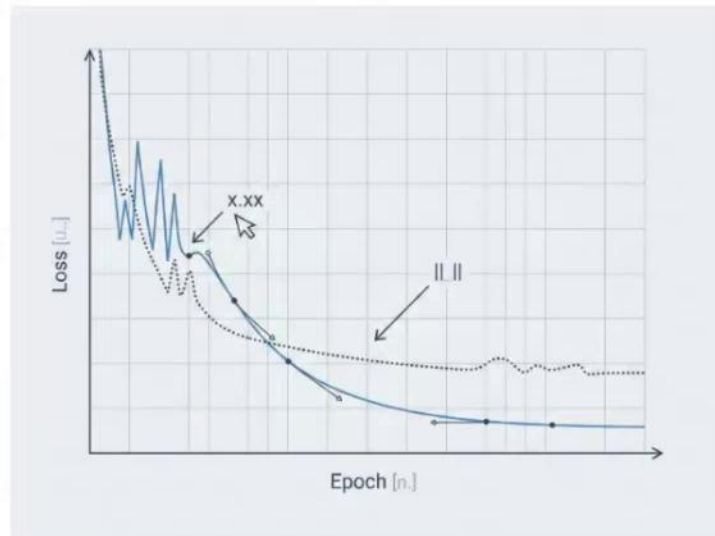


```
[1] from ultralytics import YOLO  
model = YOLO('yolov8n.pt')  
model.train(data='dataset.yaml', epochs=100)
```

Interpretación de Métricas de Entrenamiento

Evaluación del Modelo

- **mAP50 y mAP50-95:** Precisión media evaluada en distintos umbrales de IoU.
- **Pérdidas (Loss):** `box_loss` (precisión espacial) y `cls_loss` (precisión de clasificación).
- **Análisis de curvas:** Monitoreo crítico para identificar y evitar el sobreajuste (overfitting).



Métrica Destacada

mAP50-95

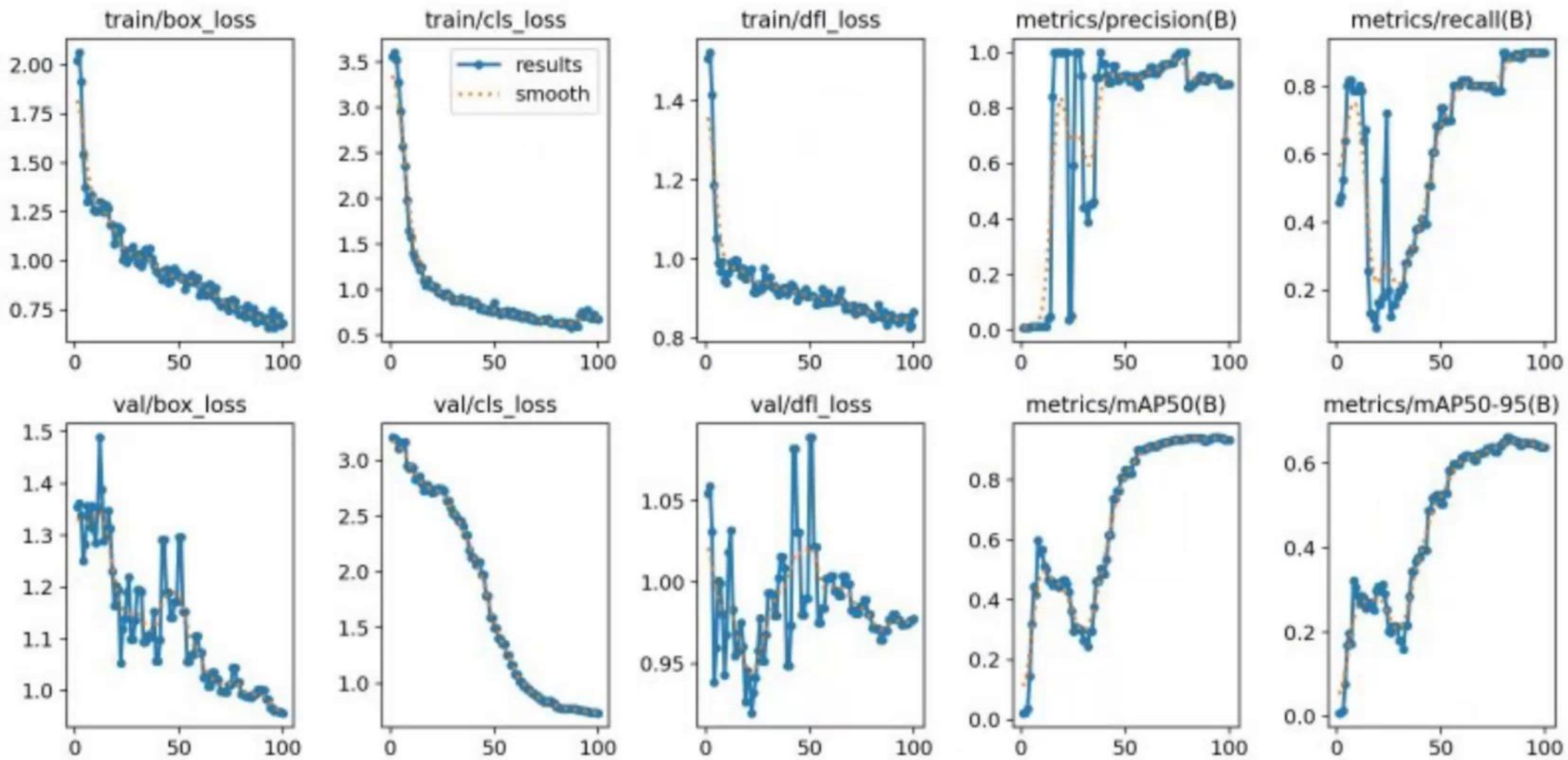
Indicador clave de precisión global evaluado en múltiples umbrales de IoU.

Matriz de confusión

- Absoluta
- Normalizada



YOLO EN VISIÓN ARTIFICIAL PARA LA INGENIERÍA



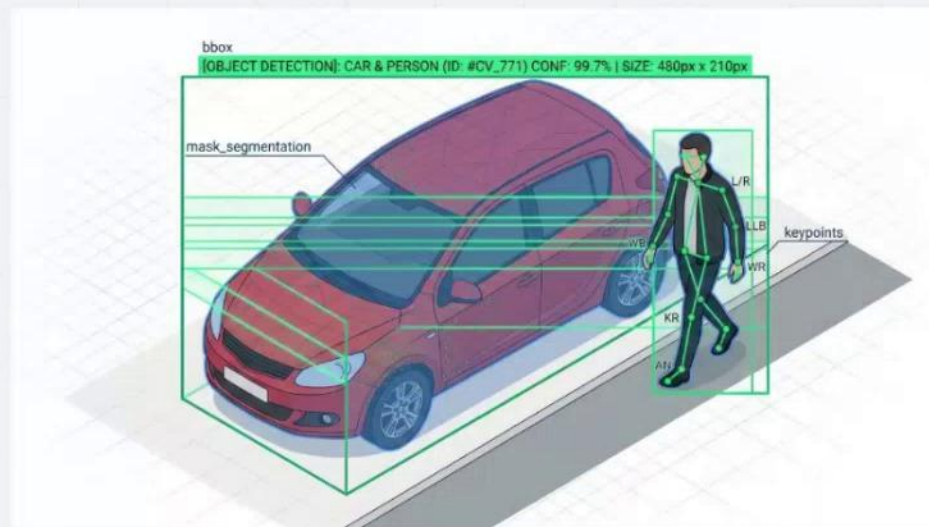
Ecosistema Multitarea de YOLO

Detección estándar: Delimitación mediante Bounding boxes.

Segmentación de instancias: Clasificación a nivel de píxel (Máscaras).

Estimación de pose: Detección de puntos clave anatómicos (Keypoints).

Capacidades adicionales: Clasificación pura de imágenes y Seguimiento (Tracking) nativo.



Visualización del Ecosistema

Inferencia unificada sobre una sola toma: Detección, segmentación y puntos clave anatómicos ejecutados en paralelo.

Selección de Escala del Modelo

• Variantes disponibles

Cinco opciones de escala para adaptarse a cada necesidad técnica: Nano (n), Small (s), Medium (m), Large (l) y XLarge (x).

• Modelos Nano / Small

Priorizan la velocidad extrema de inferencia. Optimizados para Edge computing, sistemas embebidos y dispositivos móviles de bajos recursos.

• Modelos Large / XLarge

Priorizan la precisión de detección máxima. Indicados para ejecución en servidores dedicados, entornos de nube y GPUs potentes.

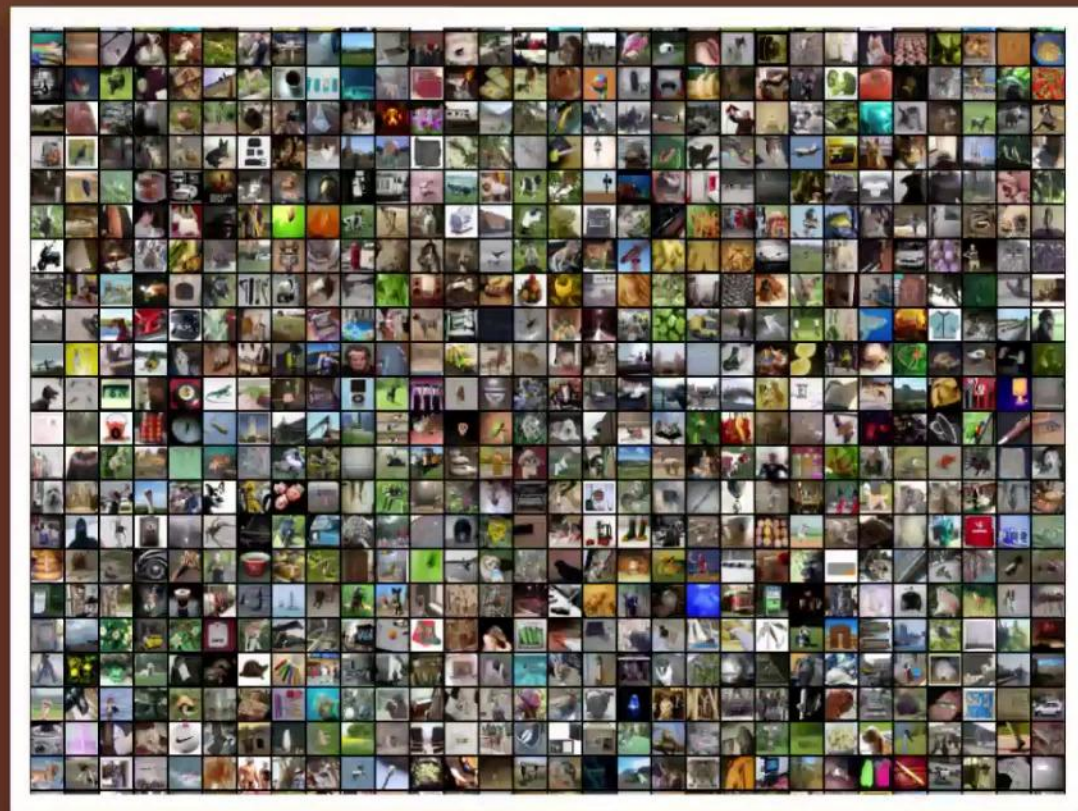
• Objetivo del diseño

Encontrar un balance técnico ideal entre latencia (tiempo de respuesta) y precisión (mAP), adaptado estrictamente al hardware objetivo.

Comparativa de Escala (YOLO)

Modelo	Complejidad / Latencia	Velocidad	Precisión
n		Ultra Rápido	Mínima
s		Rápido	Media-Baja
m		Balanceado	Media-Alta
l		Moderado	Alta
x		Baja Lat.	Máxima

* La escala afecta directamente el conteo de parámetros de la red neuronal.

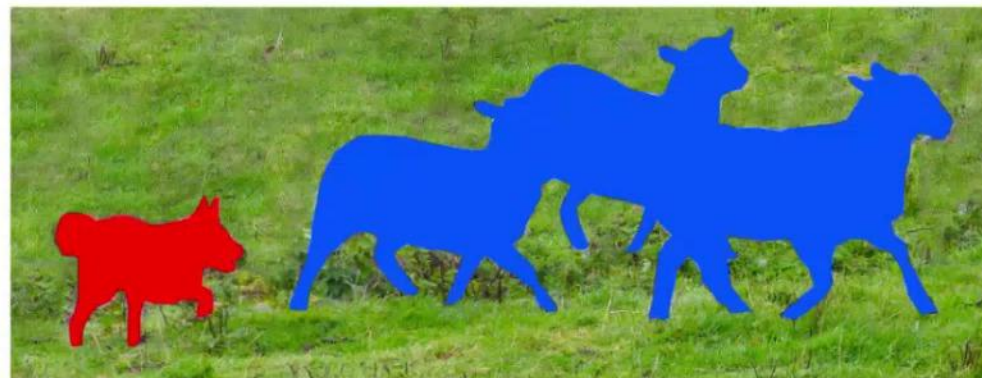


Datasets

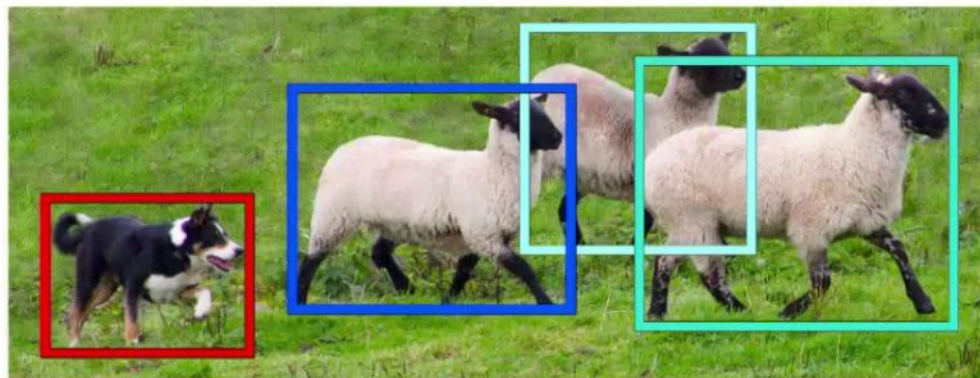
Visión artificial
Alejandro Silvestri



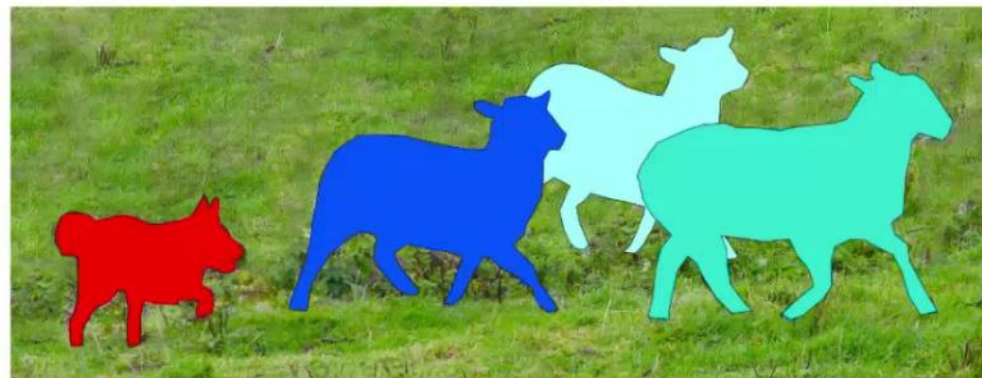
Image Recognition



Semantic Segmentation



Object Detection

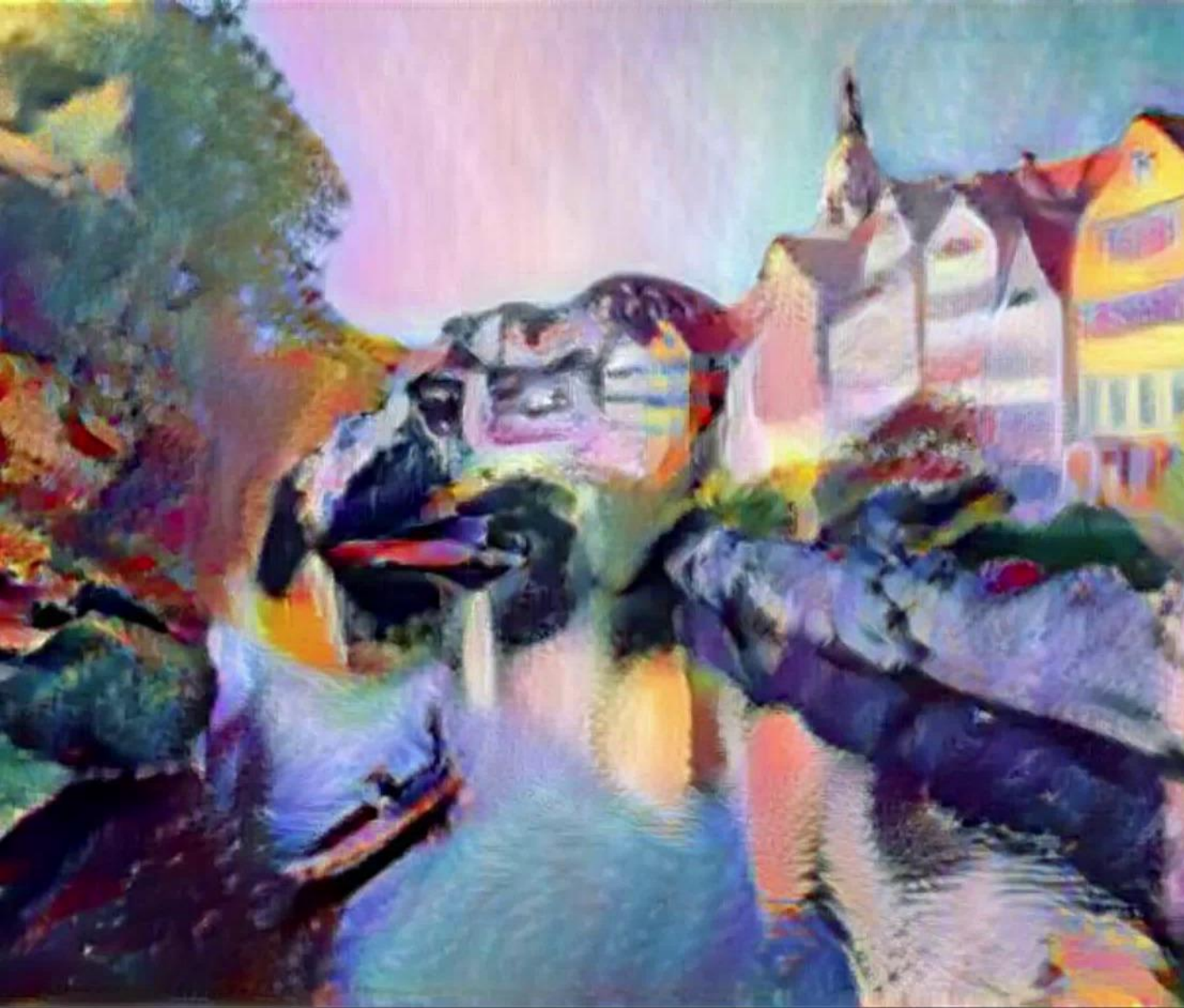


Instance Segmentation



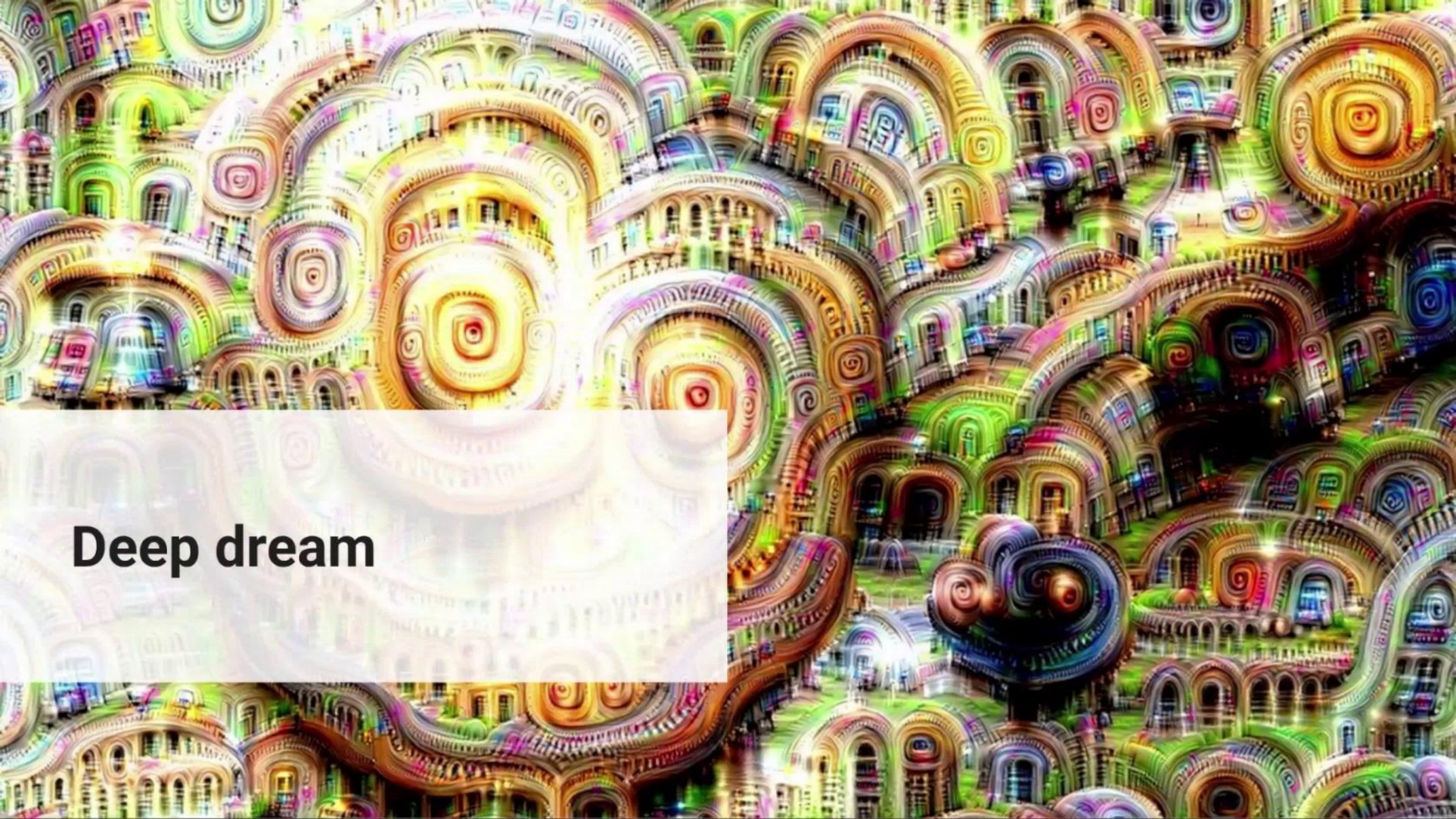
Problemas resueltos con deep learning

Alejandro Silvestri



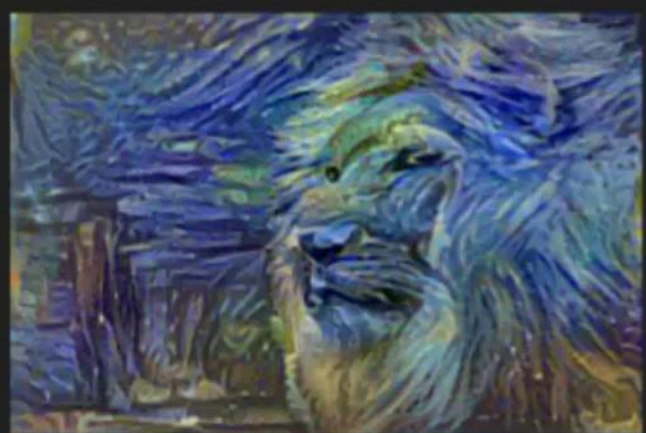
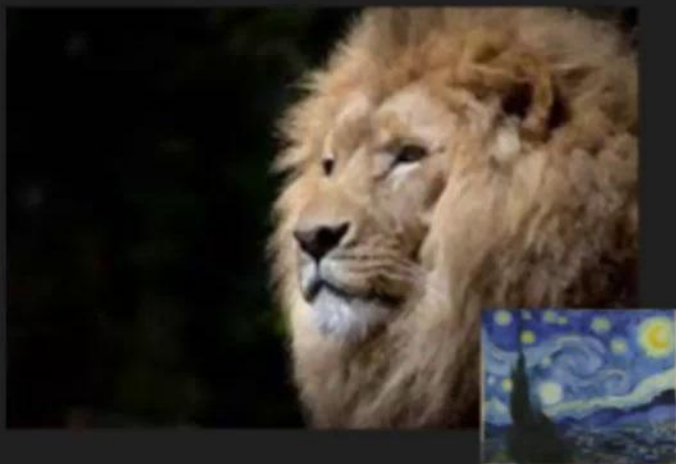
Generación de imágenes artificiales

- Deep dream
 - Style transfer
 - GAN
-
- No pertenecen al ámbito de la visión artificial, sino del procesamiento de imágenes



Deep dream

Style transfer



GAN

Generative
Adversarial
Network



Deep learning en visión artificial

- Clasificación
- Detección
- Segmentación semántica e instanciada
- Detección y reconocimiento de texto
- Convolutional Pose Machine
- Facial landmarks
- Detección de prominencias



Clasificación

- Input: una imagen
- Output
 - idealmente una etiqueta
 - en la práctica, un **nivel de confianza** para cada etiqueta

airplane

automobile

bird

cat

deer

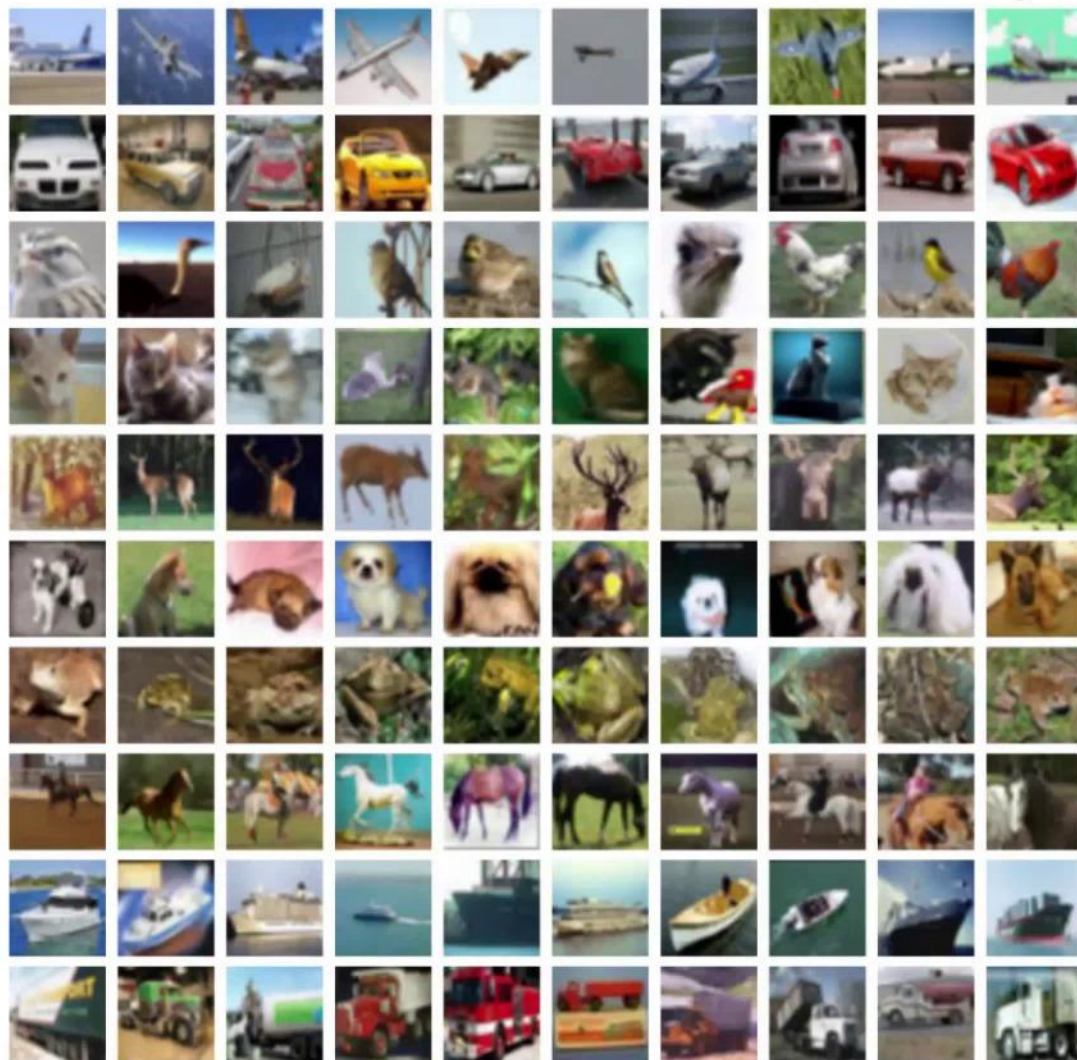
dog

frog

horse

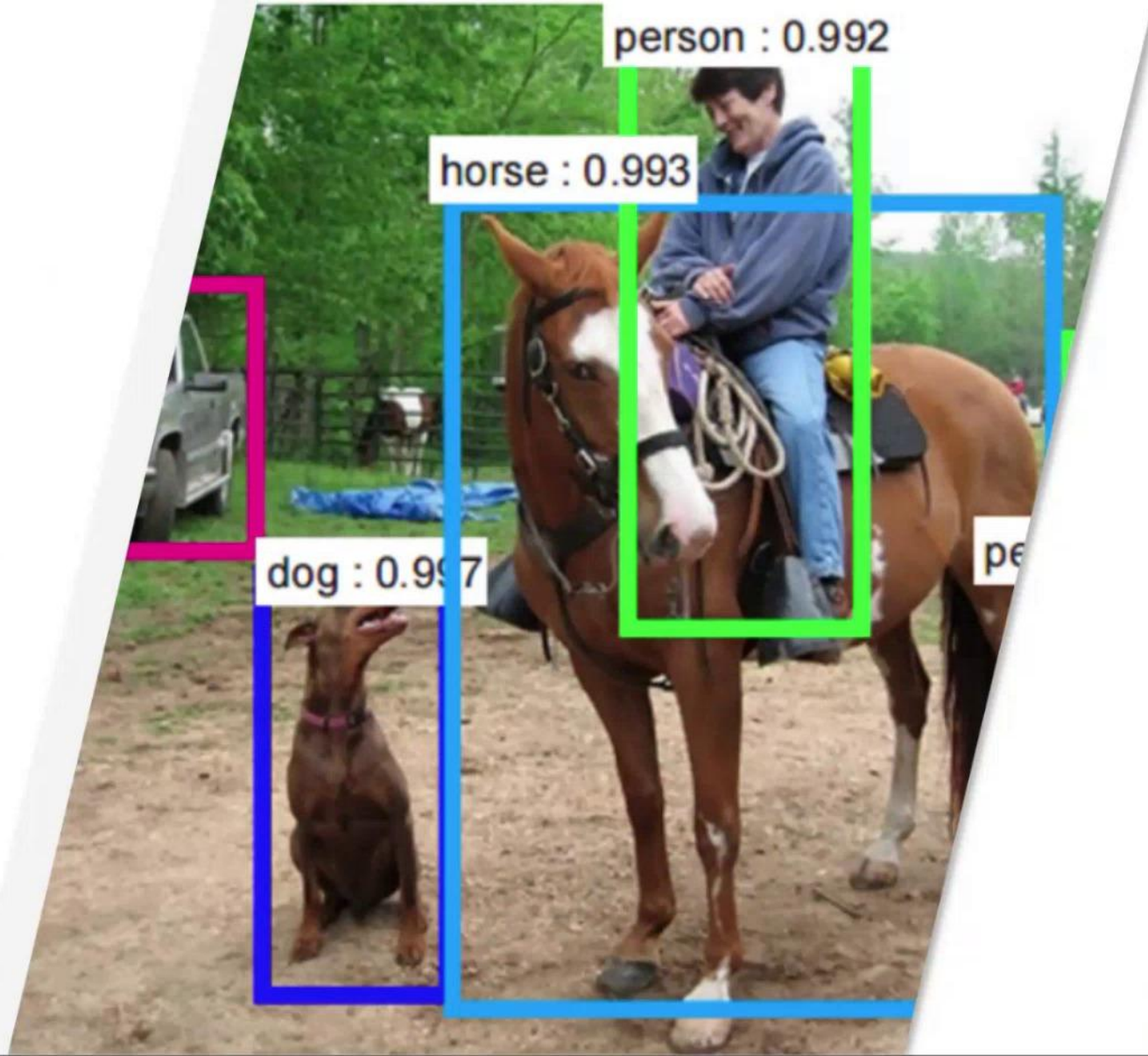
ship

truck



Detección

- El output de la red son **cajas categorizadas**
- Un detector emplea un **clasificador**



Segmentador semántico

- Un segmentador es un clasificador per pixel
- Semántico se refiere a su capacidad de clasificar por significado en lugar de hacerlo por la vecindad del pixel



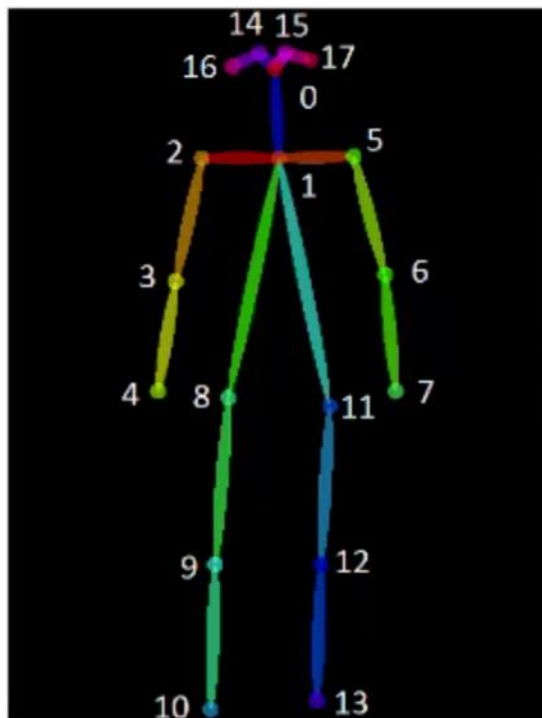
Convolutional pose machine

- Detectores de keypoints conectados
 - Producen puntos, no cajas
- Algunos detectores
 - Pose del cuerpo
 - Pose de mano



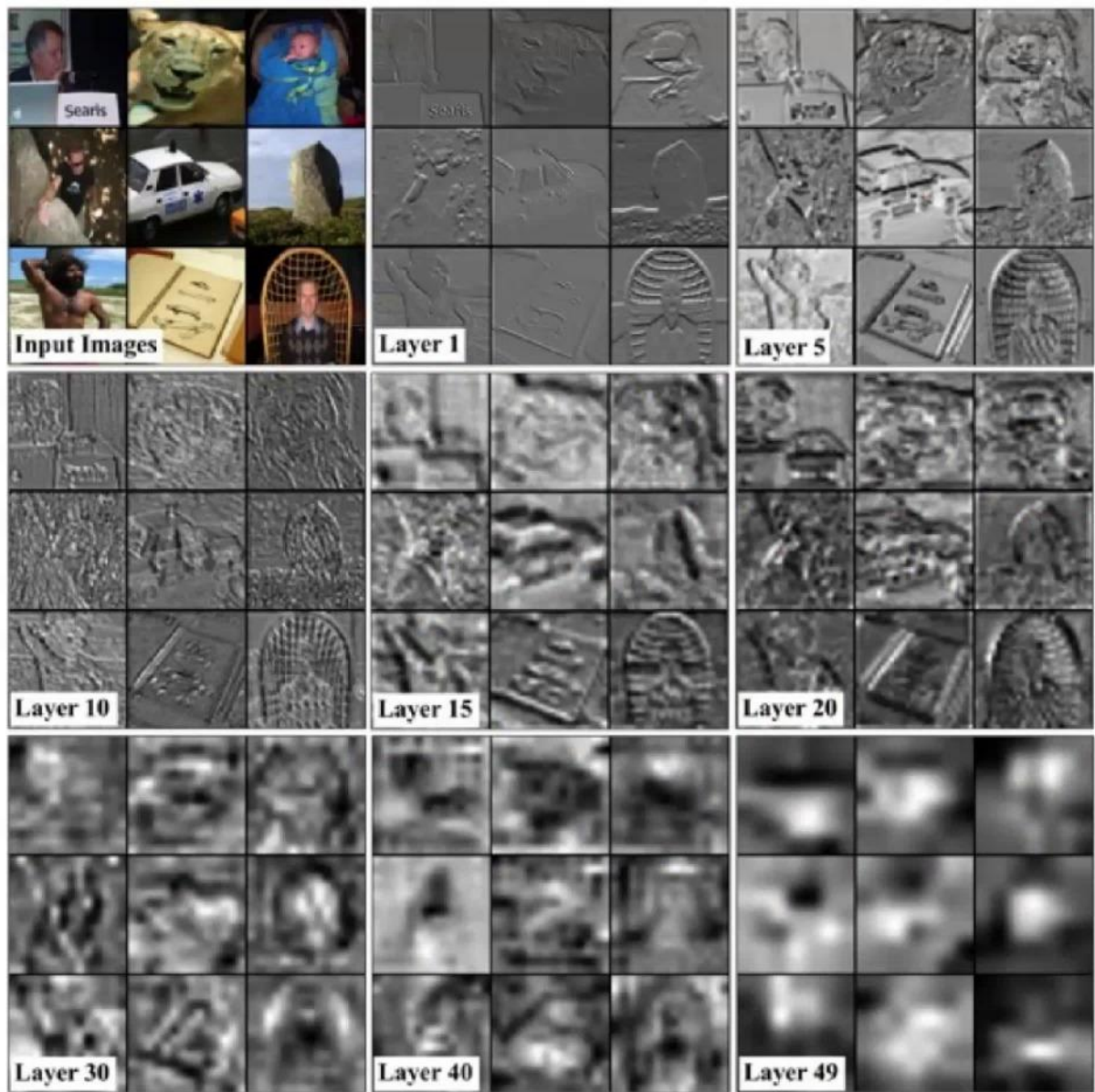
Pose del cuerpo

- 18 puntos en el dataset de COCO



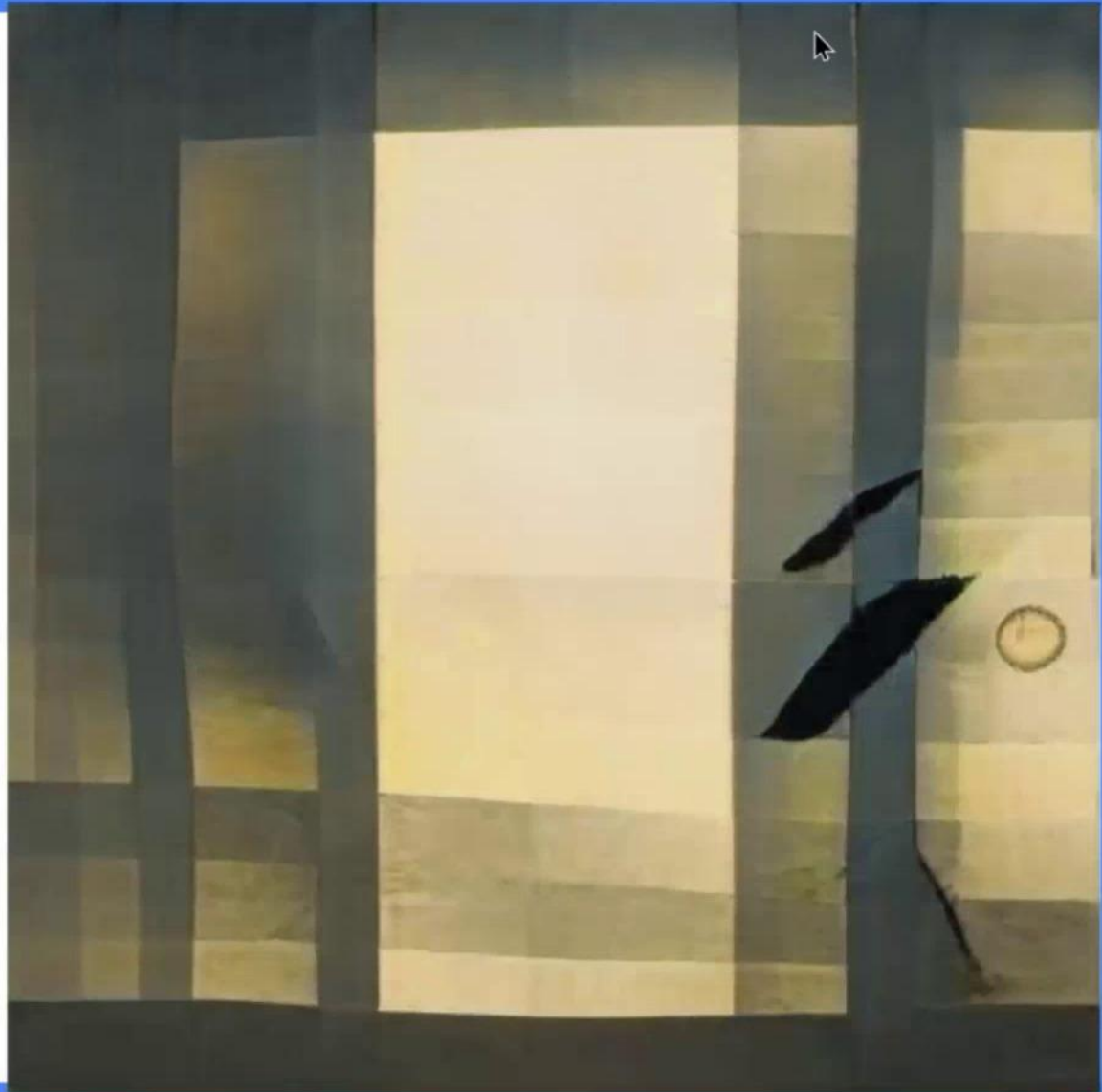
Saliency detectors: Detección de prominencias

- Prominencias son los lugares de la imagen donde la vista se posa primero
- Producen mapas de confianza
- Se usan para
 - evaluar la efectividad de publicidad y señalética
 - concentrar la atención de vehículos autónomos en señales de tránsito, peatones y otros vehículos



Referencias

- [This Person Does Not Exist](#)
- [Training a Hand Detector](#)
- [Understanding OpenPose](#)



Datasets para visión artificial

- En la jerga de machine learning, un dataset es un conjunto de datos para entrenamiento de una máquina
 - si el entrenamiento es supervisado, el dataset incluye las respuestas
- Los datasets para visión artificial son datasets de imágenes y datasets de videos
 - con las correspondientes respuestas, dependiendo del tipo de problema
- Entrenar una net requiere en el orden de 1.000 imágenes por categoría
 - **los datasets son enormes**

label = 5



label = 0



label = 4



label = 1



label = 9



label = 2



label = 1



label = 3



label = 1



label = 4



label = 3



label = 5



label = 3



label = 6



label = 1



label = 7



label = 2



label = 8



label = 6



label = 9



master chef can



cracker box



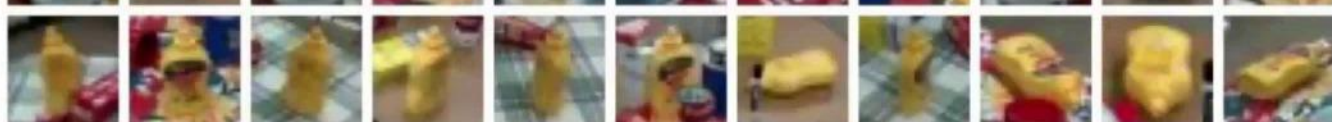
sugar box



tomato soup can



mustard bottle



tuna fish can



gelatin box



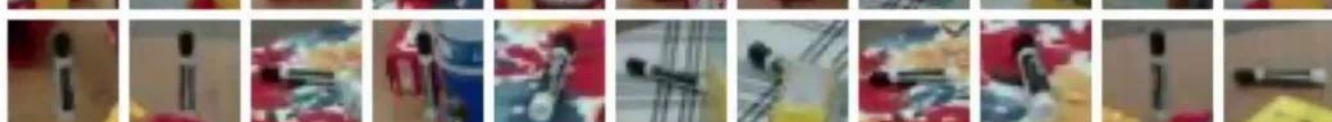
potted meat can

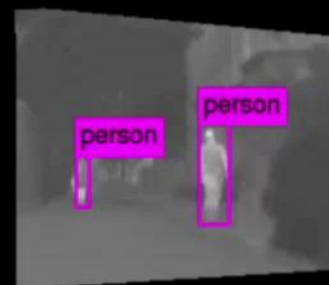
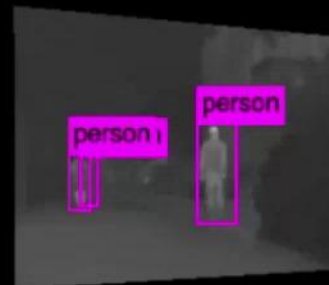
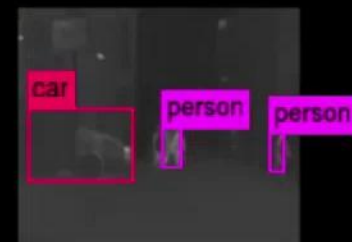
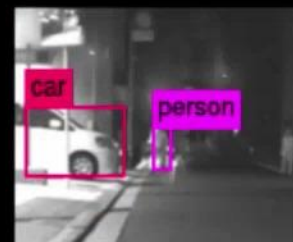
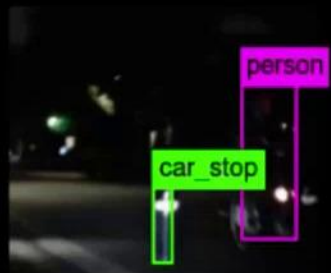


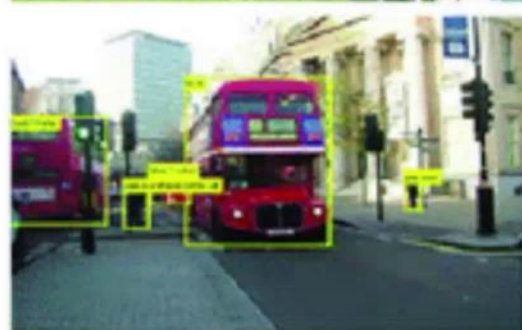
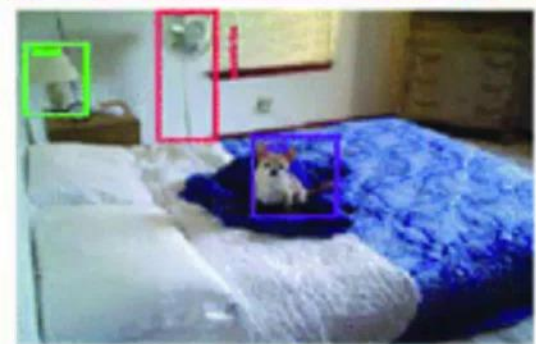
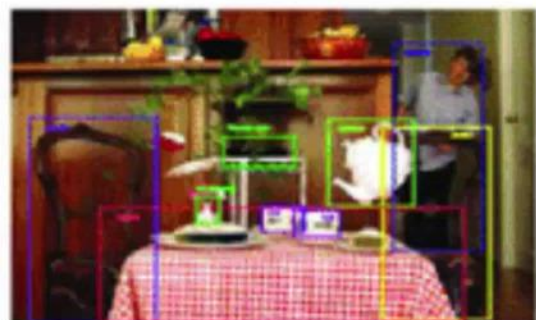
mug

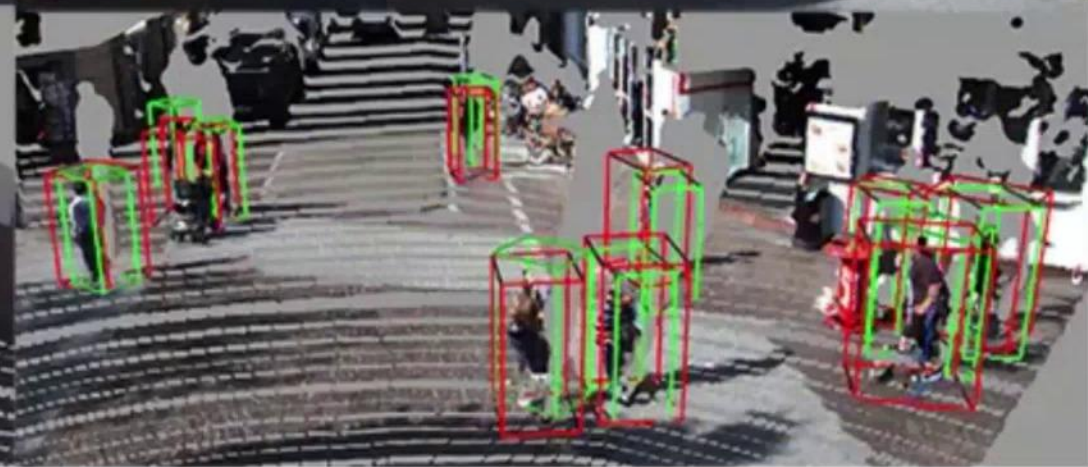
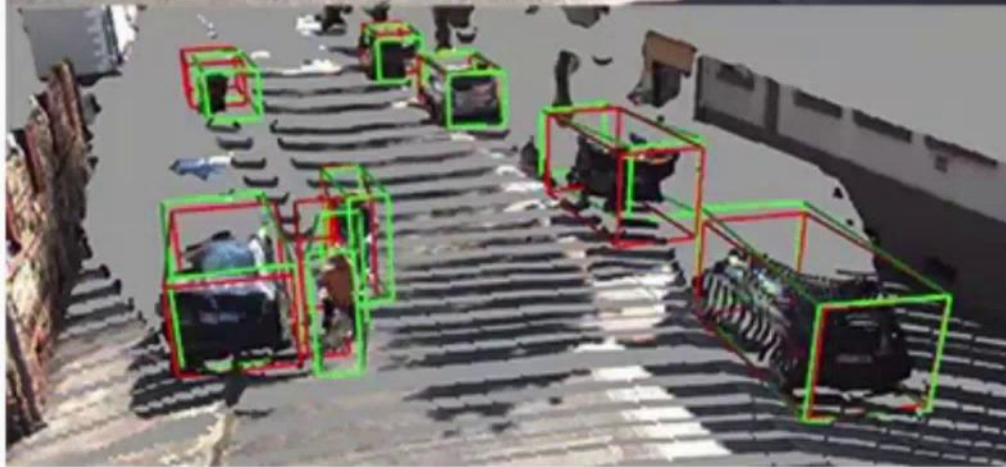
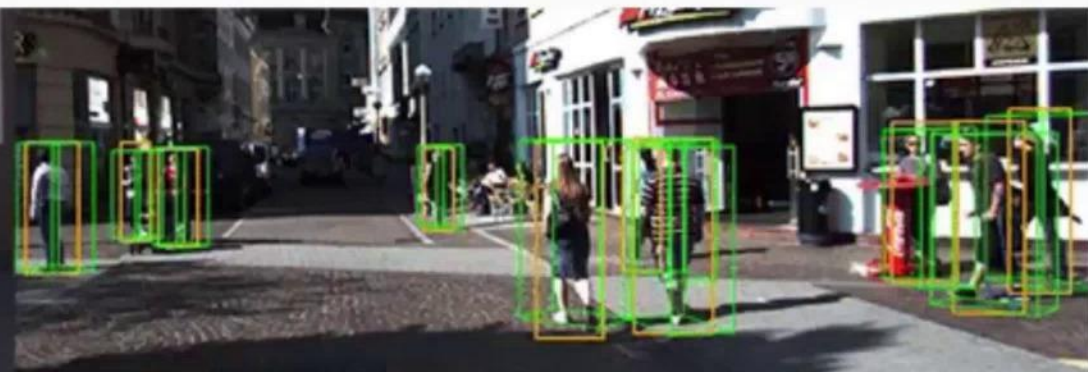
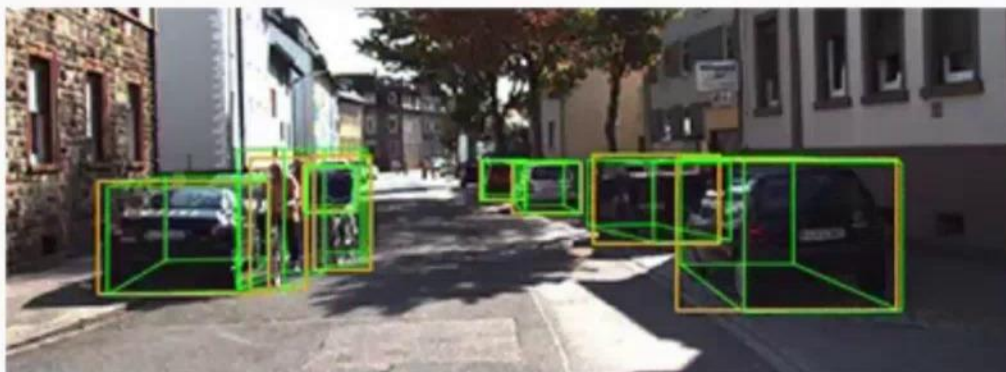


large marker



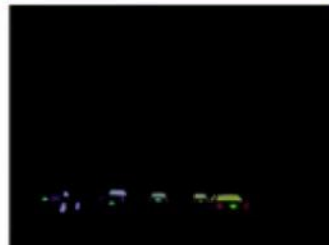
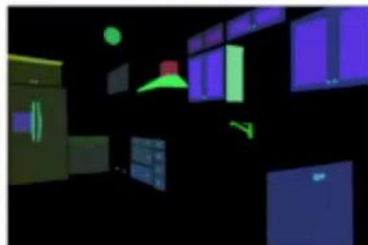
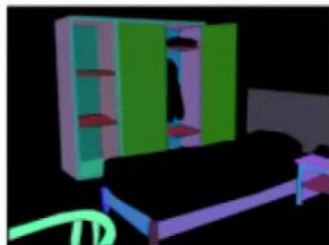
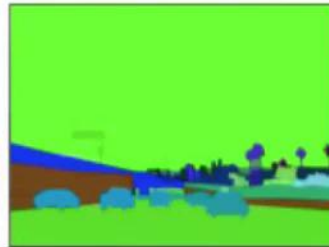
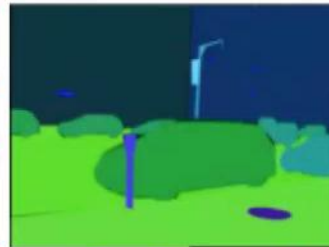
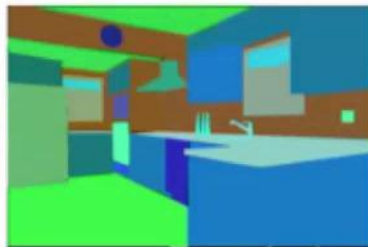
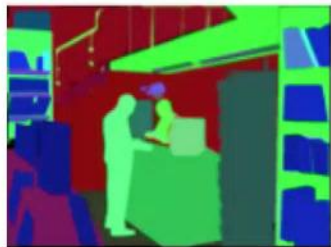








sky
 tree
 road
 water
 bldg
 mntn
 fg obj.



Qué hay en un dataset

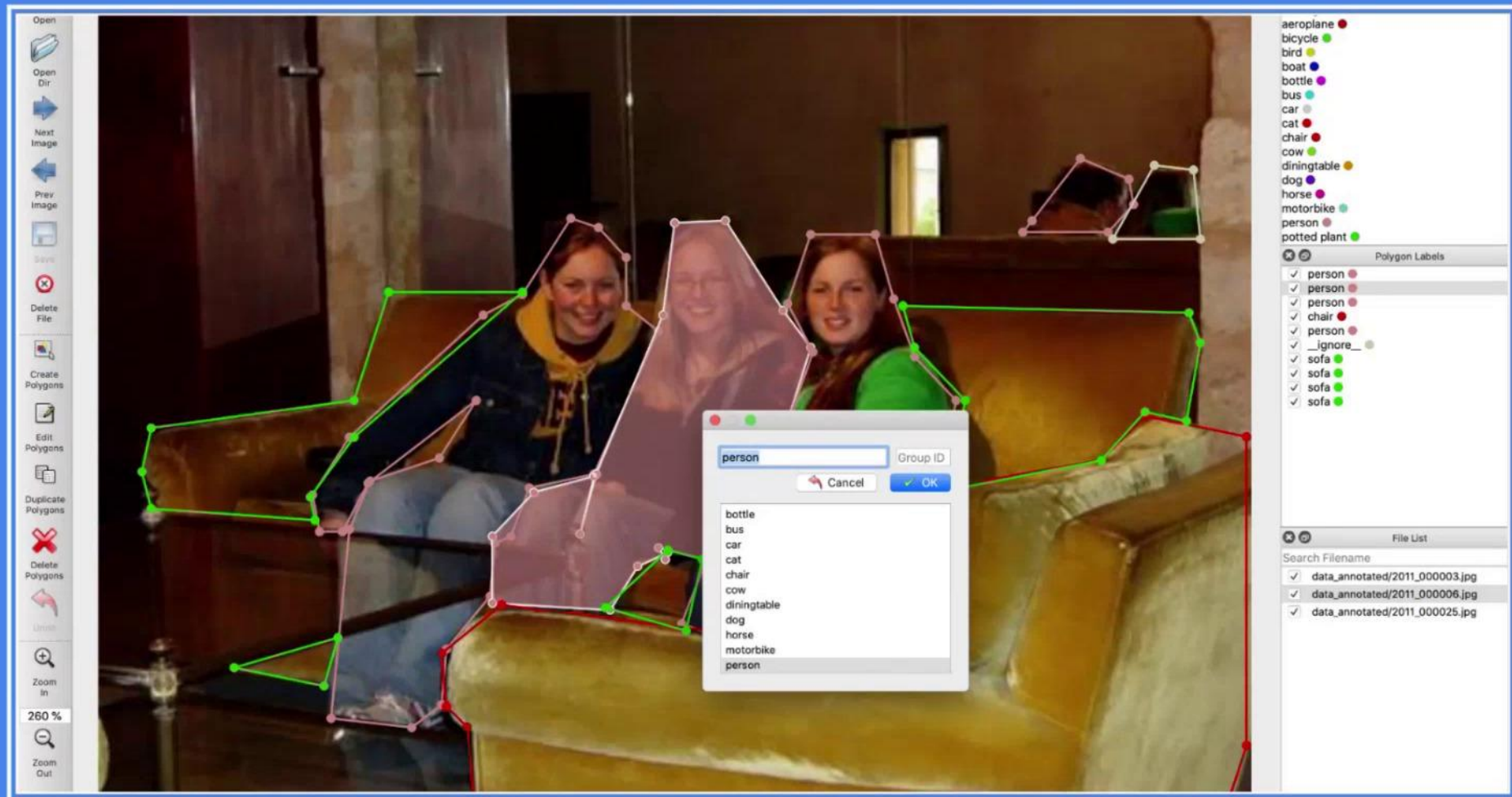
```
images/  
    2011_000001.jpg  
    2011_000002.jpg  
    2011_000003.jpg  
    ...  
annotations/  
    2011_000001.xml  
    2011_000002.xml  
    2011_000003.xml  
    ...
```

- Carpeta de imágenes
 - imágenes de un mismo tipo (JPG, PNG, etc.)
 - usualmente del mismo tamaño
 - nombradas de manera correlativa
 - **2011_000003.jpg**
- Carpeta de anotaciones
 - archivos de texto (JSON, XML)
 - nombradas con el mismo nombre de la imagen
 - **2011_000003.xml**
 - contienen las respuestas esperadas para cada imagen
- Carpeta de imágenes de segmentaciones
 - en el caso de segmentación, el entrenamiento supervisado requiere proporcionar las imágenes con la segmentación esperada

Tipos de anotaciones

```
annotation{
  "id": int,
  "image_id": int,
  "category_id": int,
  "segmentation": RLE or [polygon],
  "area": float,
  "bbox": [x,y,width,height],
  "iscrowd": 0 or 1,
}
```

- Clasificación
 - etiqueta
- Detección
 - cajas (coordenadas de vértices) etiquetadas
- Detección de keypoint
 - coordenadas de keypoints etiquetados
- Segmentación
 - imagen de etiquetas
- Estos datos se registran en el archivo de anotaciones
 - para segmentación la imagen de etiquetas se puede codificar en el archivo de anotaciones, o adjuntarse como archivo de imagen separado



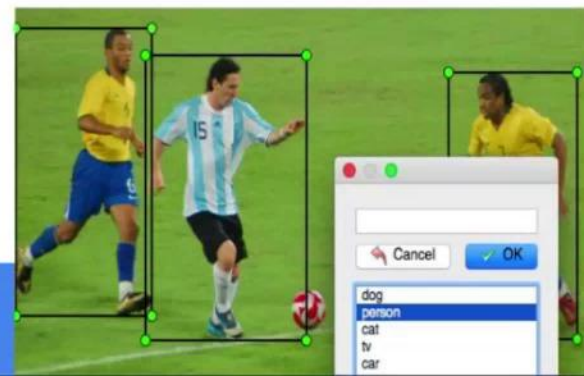
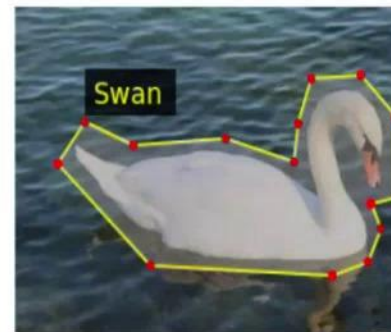
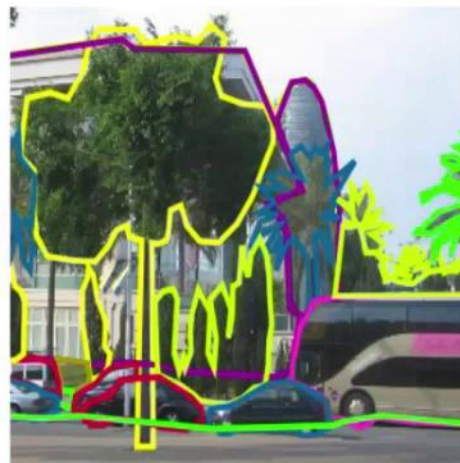
Aplicaciones de anotación

- Hay aplicaciones propietarias y de código abierto para la anotación de datasets
 - como [LabelMe](#)
- Hay servicios de anotación “humana”
 - la anotación humana se considera de mejor calidad que la anotación automática
- La anotación automática es válida en dos casos
 - si ya existe una net entrenada para hacer ese trabajo, y se está entrenando una red más eficiente, o una red que combine capacidades de varias
 - anotación semiautomática con algoritmos como grabcut, watershed, magic scissor



Aplicaciones de anotación

- [Roboflow](#)
- [Label Studio](#)
- [MakeSense.AI](#)
- [LabelImg](#)
- [VGG image annotator](#)
- [LabelMe](#)
- [Scalable](#)
- [RectLabel](#)
- [Supervisely \(Artículo\)](#)
- [coco-annotator: :pencil2](#)
- [Artículo sobre formas de anotación](#)



Principales aplicaciones

<> Locales y de código abierto

CVAT — promocionado por OpenCV

Label Studio — evolución de LabelMe

☁ SaaS, plataforma de anotación a entrenamiento

Roboflow — el acceso gratuito exige hacer público el dataset y limita la cantidad de imágenes

Encord, V7 Darwin — varios especializados

👥 Corporativos

Para trabajo en grandes equipos



Aumentación

- Las redes requieren gran variedad de imágenes para su entrenamiento
- Parte de esta variedad se puede generar artificialmente por procesamiento de imágenes, produciendo también la anotación correspondiente
 - cambio de brillo y contraste
 - espejo, rotación
 - zoom (recorte y aumento de tamaño)
 - deformación
 - relación de aspecto
 - deformación elástica
- Estas técnicas permiten multiplicar la cantidad de imágenes de un dataset
 - se consideran útiles pero de inferior calidad
 - se mantiene trazabilidad que permitan identificar si una imagen es original, y si no qué proceso se usó para generarla



IMAGE LEVEL AUGMENTATIONS



Flip



90° Rotate



Crop



Rotation



Shear



Brightness



Exposure



Blur



Noise



Cutout



Mosaic

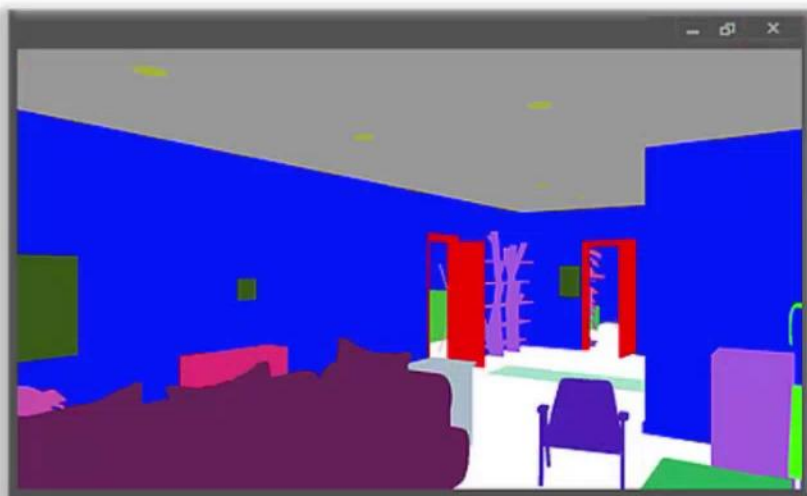
Aplicaciones de aumentación

- El software de anotación suele tener funciones de aumentación; se elige
 - qué tipos de transformaciones se quieren usar
 - rangos para generar parámetros aleatorios
 - factor de aumentación (típicamente 5)
 - producen un dataset aumentado de imágenes y anotaciones correspondientes
- Los frameworks de desarrollo suelen tener funciones de aumentación
 - operan de manera similar
 - producen imágenes y anotaciones aumentadas para entrenar inmediatamente, y no se guardan



Dataset sintético

- Imagen sintética
 - imagen obtenida de un mundo virtual 3D
- Hay software para generar estas imágenes virtuales
 - también producen la anotación correspondiente
 - incluyendo las imágenes de etiquetas para segmentación
- Las imágenes sintéticas se consideran de inferior calidad que las reales
 - un gran porcentaje de imágenes sintéticas confunden a las redes, que luego no reconocen imágenes reales



Dataset privado

- Los problemas especializados requieren la elaboración de un dataset ad hoc
- Internet es una enorme fuente de imágenes que se utiliza para crear estos datasets
- Cuando la temática es muy especializada y no se encuentran imágenes adecuadas en tipo y cantidad en Internet, hay que tomar fotografías
- Luego de reunir las imágenes es necesario anotarlas
 - etiquetarlas para clasificación
 - dibujar bordes para segmentación
 - dibujar cajas para detección
- Requiriendo en el orden de **1.000 imágenes por categoría**, la elaboración del dataset se convierte en el problema más importante
 - por eso quienes elaboran un dataset privado no lo comparten

¿Por qué hay datasets públicos?



- Los datasets públicos se crean para incentivar el desarrollo tecnológico para determinados problemas que resultan inviables hacerlos de manera particular
 - a veces se requiere apoyo comunitario para construir el dataset
 - a veces se requiere que la comunidad de visión artificial desarrolle una solución para ese dataset
- Sobre ellos se organizan competencias de visión artificial

IMGENET

- [ImageNet](#)
- El mayor dataset para objetos
 - 14M de objetos anotados a mano
 - etiquetados, descritos y con features SIFT
 - 20.000 categorías





COCO

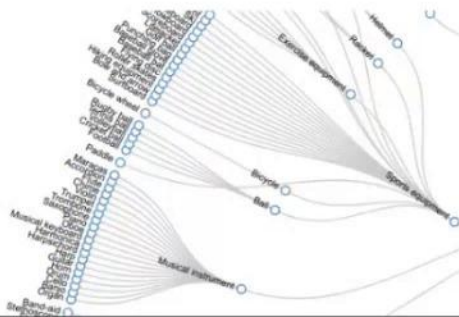
Common Objects in Context

- [COCO](#)
- El dataset de objetos comunes más popular
 - 90 categorías
 - 1,5M instancias de objetos
 - 200.000+ imágenes etiquetadas
 - 100.000 no etiquetadas
- Respaldo por Microsoft y Facebook



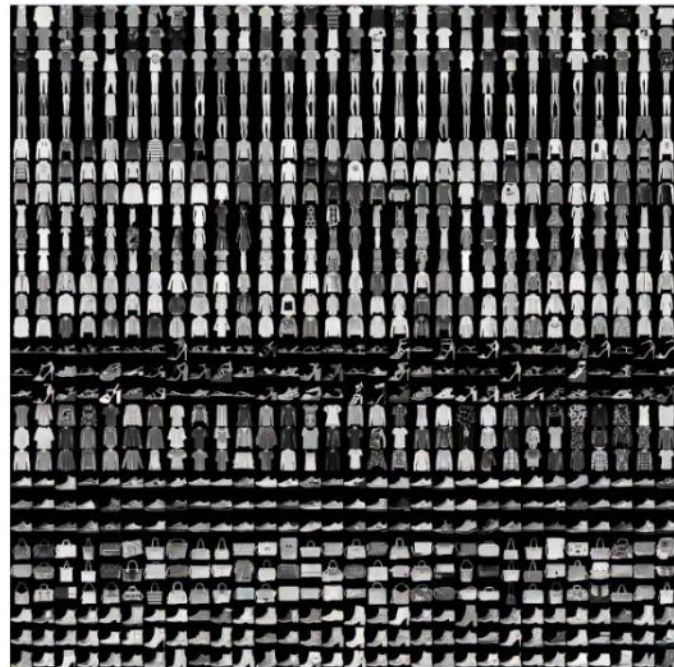
Open Image

- [OpenImage](#)
- Gran dataset anotado
 - Más reciente que COCO
 - 9M de imágenes con etiquetas
 - 16M de cajas en 2M de imágenes
 - 600 [categorías](#)
- Respaldo por Google



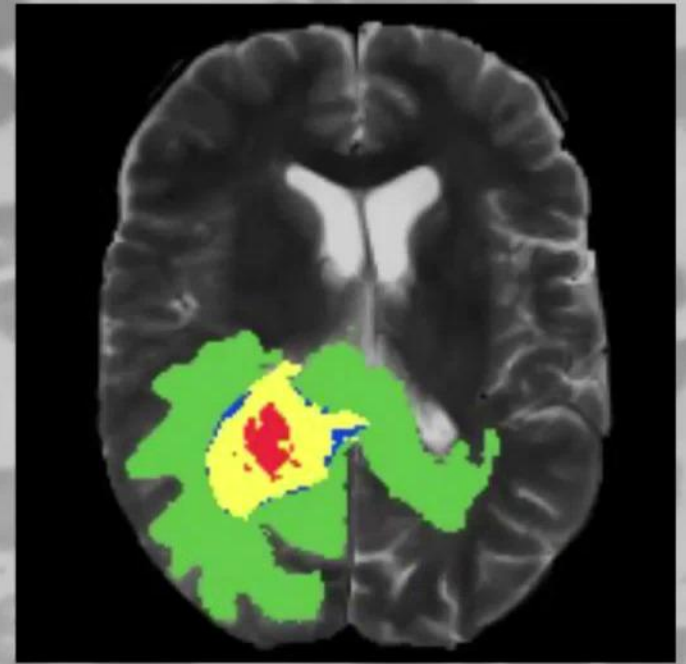
Datasets clásicos

- [MNIST handwritten digits](#)
 - 70.000 fotos grises de dígitos manuscritos
- [Fashion-MNIST](#)
 - 70.000 fotos grises de ropa
 - clasificadas en 10 categorías
- MNIST database
 - *Modified National Institute of Standards and Technology database*

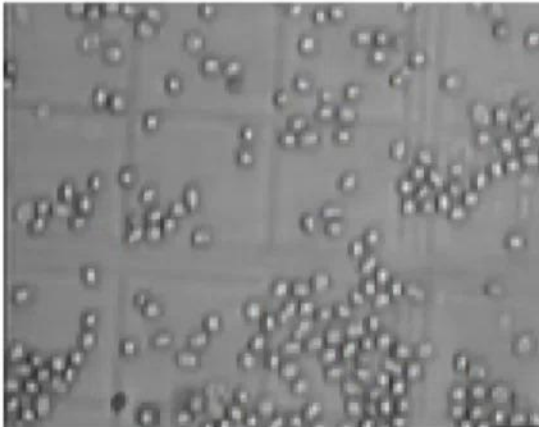


Datasets biomédicos

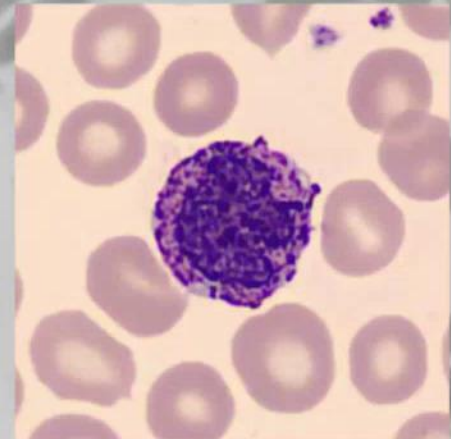
- Imágenes de
 - microscopio, microscopio electrónico
 - ecografía, ecodoppler
 - resonancia magnética



A (13).bmp Cropped image



No. of RBCs=187



Lista de etiquetas

- El listado de etiquetas (labels, categorías, clases) es propio de un dataset
 - un mismo dataset puede tener más de un listado
 - COCO comenzó con 20 categorías y hoy tiene 90
- Suele ser un pequeño archivo de texto con los nombres de las categorías encolumnados
 - este archivo es imprescindible para interpretar el resultado de un clasificador, de un detector y de un segmentador semántico

```
1: 'person',
2: 'bicycle',
3: 'car',
4: 'motorcycle',
5: 'airplane',
6: 'bus',
7: 'train',
8: 'truck',
9: 'boat',
10: 'traffic light',
11: 'fire hydrant',
13: 'stop sign',
14: 'parking meter',
15: 'bench',
16: 'bird',
17: 'cat',
18: 'dog',
19: 'horse',
20: 'sheep',
21: 'cow',
22: 'elephant',
23: 'bear',
24: 'zebra',
25: 'giraffe',
27: 'backpack',
28: 'umbrella',
31: 'handbag',
32: 'tie',
33: 'suitcase',
34: 'frisbee',
35: 'skis',
36: 'snowboard',
37: 'sports ball',
38: 'kite',
39: 'baseball bat',
40: 'baseball glove',
41: 'skateboard',
42: 'surfboard',
43: 'tennis racket',
44: 'bottle',
46: 'wine glass',
```



Visión artificial
Alejandro Silvestri

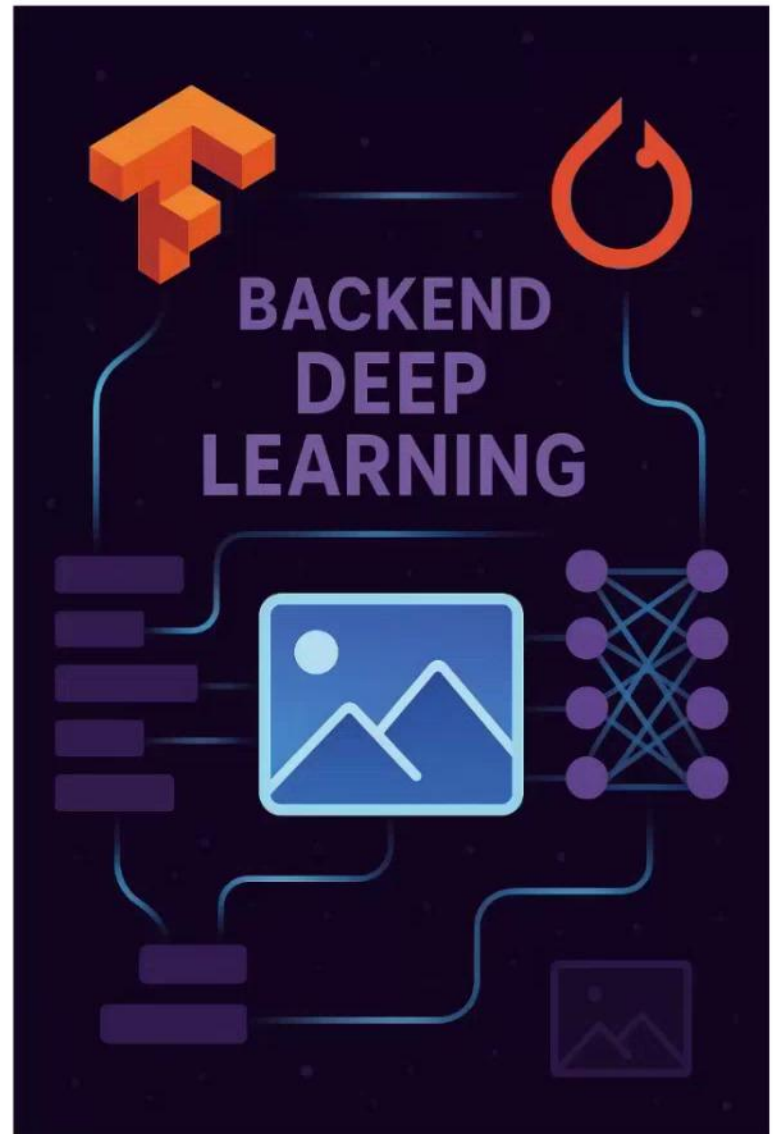
Frameworks de
deep learning



Framework de deep learning

es un conjunto de herramientas para entrenamiento e inferencia, que se deben instalar en el dispositivo del desarrollador, usualmente compuesto de

- **frontend**, bibliotecas para el desarrollador
- **backend**, donde se ejecutan los modelos
- **aplicaciones**



Qué se espera de un framework

- multiplataforma y eficiencia computacional
 - rápido para inferir
 - rápido para entrenar
- compatibilidad con modelos existentes
 - requiere soporte para todos los tipos de capas
 - nuevas capas surgen cada año
- herramientas y API
 - herramientas para entrenamiento
 - API para entrenamiento e inferencia, en varios lenguajes (principalmente Python)

Frameworks de deep learning

- principales, dominantes
 - **TensorFlow 2**
 - **PyTorch**
- otros menores
 - darknet (independiente)
 - MXNet (Amazon)
- obsoletos
 - Caffe (Berkeley university)
 - Torch (Facebook)
 - CNTK (Microsoft)
 - y muchos más



Caffe

Diáspora y convergencia

- 2012 - AlexNet
 - red convolucional fundacional, sin framework (desarrollo propio)
- 2014 a 2017 - Caffe
 - primer framework para deep learning, inspirado en AlexNet
 - el más influyente
- 2015 - **Diáspora**
 - nuevos frameworks siguiendo el camino de Caffe
 - varias bibliotecas dieron soporte a deep learning (Torch, DLib, Keras, DarkNet)
 - todos intentan hacer lo mismo
- 2017 a 2019 - **Unificación**
 - con **TensorFlow** 1.0 se inicia una unificación de frameworks que concluye en 2019
- 2019 - **Especialización**
 - se consolidan frameworks especializados en inferencia, en particular para **edge computing**

Frontend

- API para el desarrollador
- Biblioteca
 - Python, Javascript, C++, o cualquier otro lenguaje
 - presenta las funciones y clases para definir, entrenar y usar modelos de deep learning



- Principales frontends
 - **Keras**
 - frontend Python de TensorFlow
 - **PyTorch**
 - frontend Python de PyTorch
- Otros
 - módulo **dnn** de OpenCV
 - sólo para inferencia, no para entrenamiento
 - pensado para usar en proyectos con OpenCV modelos ya entrenados, sin necesidad de cambiar de frontend



```
import tensorflow as tf
mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation=tf.nn.relu),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation=tf.nn.softmax)
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

The Keras logo, featuring a red square with a white letter 'K' inside.

Keras

```
import torch
import torch.nn as nn
import torch.optim as optim
```

```
# Define the neural network
```

```
class ConvNet(nn.Module):
```

```
    def __init__(self):
```

```
        super(ConvNet, self).__init__()
```

```
        self.conv1 = nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2)
```

```
        self.bn1 = nn.BatchNorm2d(32)
```

```
        self.relu1 = nn.ReLU()
```

```
        self.maxpool1 = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
        self.conv2 = nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2)
```

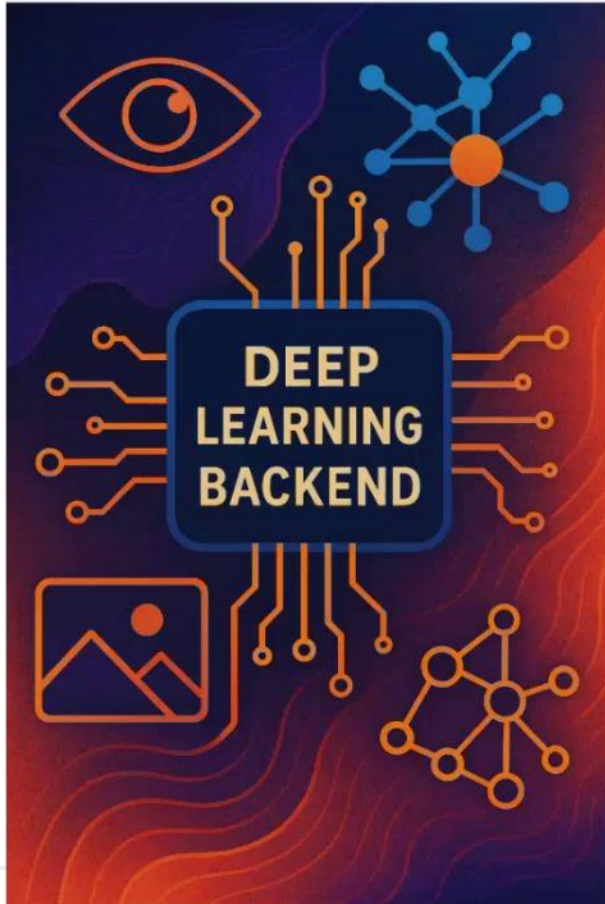
```
        self.bn2 = nn.BatchNorm2d(64)
```

```
        self.relu2 = nn.ReLU()
```

```
        self.maxpool2 = nn.MaxPool2d(kernel_size=2, stride=2)
```

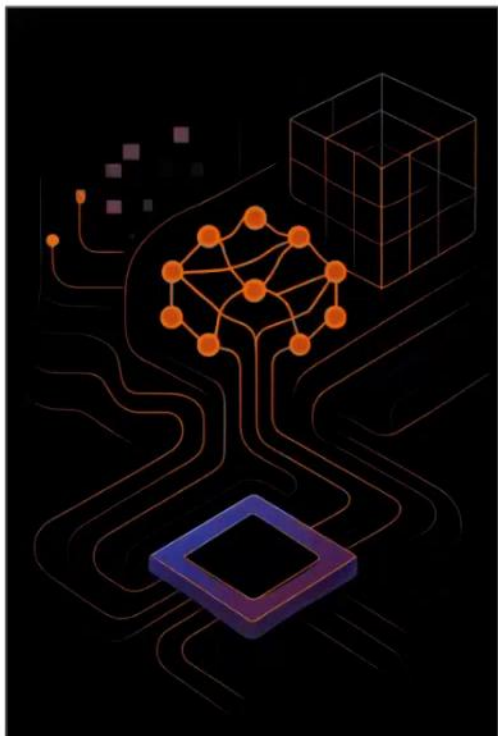
PYTORCH

Backend



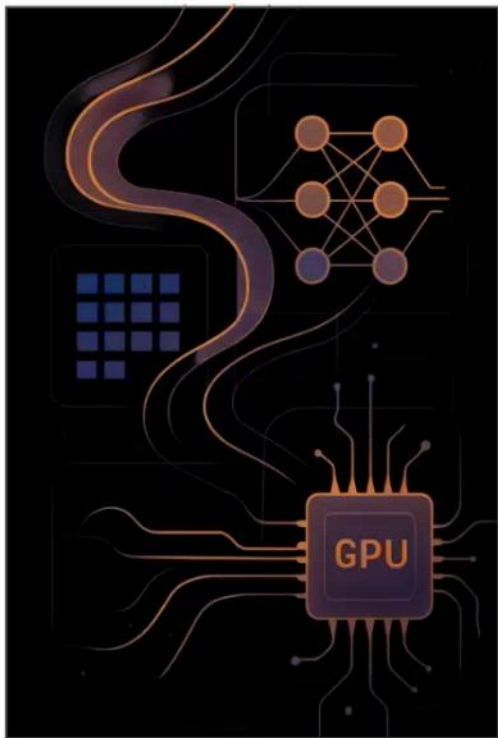
- *Código que ejecuta el modelo de deep learning*
- Debe estar instalado en el dispositivo
 - para predicción
 - para entrenamiento
- Dominantes
 - **TensorFlow**
 - **PyTorch**
- OpenCV dnn tiene un backend propio para CPU, y admite usar otros backends

Backend de inferencia



- *para ejecutar modelos ya entrenados*
- TensorFlow Lite
 - backend liviano
 - para móviles
 - para modelos livianos
- OpenVINO
 - para computación edge
 - para chips Intel
- ONNX runtime

Al backend se le pide eficiencia computacional



- El backend debe aprovechar
 - todo el hardware disponible
 - CPU, GPU, TPU, NPU
- Implementación eficiente
 - de los algoritmos que ejecutan las capas de las redes
- Usa drivers y bibliotecas de interfaz del hardware
 - CUDA, cuDNN, ROCm, oneDNN, OpenCL...

Edge computing

- Computación de borde
 - es la que se ejecuta en o cerca de la cámara
 - abarca celulares y cámaras con chipsets de AI
- Chipsets de AI
 - como Intel Movidius Myriads X
 - son microprocesadores de bajo consumo y especializados, muy adecuados para ejecutar modelos de deep learning
- Los frameworks para edge computing suelen compilar en dos pasos
 - representación intermedia
 - la compilación pesada en PC logra un código optimizado independiente de la plataforma de ejecución
 - motor de inferencia
 - framework específico para la plataforma, que ejecuta la representación intermedia



2 formas de implementar un modelo

- programáticamente
 - **Keras**
 - **PyTorch**
- archivo de net configurations
 - muy práctico para distribución
 - fácil de convertir de un formato a otro

Modelo

- Net configurations
 - Archivo que describe uno a uno los layers de la DNN
 - A la derecha, `deploy.prototxt` del framework **Caffe**
 - ocupa **24 KB**
- Weights
 - Ponderaciones de las neuronas ajustadas durante el aprendizaje
 - Ejemplo: `res10_300x300_ssd_iter_140000.caffemodel`
 - Ocupa **10 MB**

```
input: "data"
input_shape {
  dim: 1
  dim: 3
  dim: 300
  dim: 300
}

layer {
  name: "data_bn"
  type: "BatchNorm"
  bottom: "data"
  top: "data_bn"
  param {
    lr_mult: 0.0
  }
  param {
    lr_mult: 0.0
  }
  param {
    lr_mult: 0.0
  }
}

layer {
  name: "data_scale"
  type: "Scale"
  bottom: "data_bn"
  top: "data_bn"
  param {
    lr_mult: 1.0
    decay_mult: 1.0
  }
  param {
    lr_mult: 2.0
    decay_mult: 1.0
  }
  scale_param {
    bias_term: true
  }
}
```

Enlaces

- Referencia
 - [Comparison of deep-learning software](#)
 - [Deep learning basics in Colaboratory](#)
- Sistemas y formatos
 - Caffe, <http://caffe.berkeleyvision.org/>
 - TensorFlow, <https://www.tensorflow.org/>
 - Torch, <http://torch.ch/>
 - Darknet, <https://pjreddie.com/darknet/>
 - DLDT, <https://software.intel.com/opencvino-toolkit>
 - ONNX, <https://onnx.ai/>